Atelier 7 : GitLab CI/CD + SonarQube Cloud

Rapport de réalisation — analyse automatique de la qualité et de la sécurité d'un mini-site web

Technologies : HTML, CSS, JavaScript, GitLab CI/CD, SonarQube Cloud, GitLab Pages

Date : 9 novembre 2026

1. Objectif

Le but de cet atelier était de mettre en place un petit projet web avec un pipeline GitLab capable de vérifier automatiquement le code avant sa publication. J'ai utilisé SonarQube Cloud pour analyser le projet et contrôler le Quality Gate.

Pour voir concrètement la différence, j'ai travaillé avec une première version correcte (V1), puis une deuxième version dans laquelle j'ai volontairement introduit une faille XSS. L'idée était de vérifier si le pipeline allait bloquer la mise en ligne du code vulnérable.

2. Outils utilisés

Outil	Utilisation
GitLab	Dépôt du projet et gestion des pipelines.
GitLab CI/CD	Lance automatiquement les jobs après un push.
SonarQube Cloud	Analyse du code et calcul du Quality Gate.
GitLab Pages	Publie le site lorsque le pipeline passe.
Git / VS Code	Modification du code et envoi des changements.

3. Organisation du pipeline

Stage	Job	Rôle
quality	sonarqube-check	Lance l'analyse SonarQube et attend le résultat du Quality Gate.
quality (V2)	syntax-check	Vérifie la syntaxe de `script.js` avec Node.
security (V2)	security-check	Vérifie les vulnérabilités et hotspots restants via l'API SonarQube.
deploy	deploy-pages	Publie les fichiers sur GitLab Pages si les étapes précédentes sont validées.

Le point important est que `sonar.qualitygate.wait=true` fait attendre le résultat de SonarQube. Si le Quality Gate échoue, le job d'analyse échoue aussi et le déploiement ne continue pas.

4. Mise en place

1) Création du projet. J'ai créé le dépôt public `atelier-sonarqube`, puis je l'ai cloné en local. La V1 contenait simplement `index.html`, `script.js` et `style.css`.

2) Connexion à SonarQube Cloud. J'ai relié SonarQube à GitLab, créé l'organisation et importé le projet avec un token d'accès.

3) Configuration de l'analyse. J'ai désactivé l'Automatic Analysis pour éviter d'avoir deux analyses en parallèle. Le fichier `sonar-project.properties` contient notamment la clé du projet, l'organisation, les sources et l'option d'attente du Quality Gate.

4) Variables CI/CD. Le token SonarQube a été ajouté dans les variables GitLab sous `SONAR_TOKEN`, avec `SONAR_HOST_URL`. Les secrets ne sont donc pas écrits directement dans le dépôt.

5) Pipeline. Enfin, j'ai ajouté le fichier `.gitlab-ci.yml` et j'ai envoyé la V1 sur GitLab.

Extrait du job d'analyse

```
sonarqube-check: stage: quality image: name: sonarsource/sonar-scanner-cli:latest entrypoint: [""]
script: - sonar-scanner -Dsonar.host.url="$SONAR_HOST_URL" -Dsonar.token="$SONAR_TOKEN"
-Dsonar.qualitygate.wait=true
```

5. Test avec la V1

La première version était volontairement simple. Un bouton modifiait le texte d'un paragraphe avec `textContent`. Après le push, SonarQube n'a signalé aucun problème : le Quality Gate était Passed avec 0 issue et le site pouvait être publié.

6. Introduction de la faille XSS

Pour tester le comportement du pipeline, j'ai ensuite modifié le JavaScript. La première tentative utilisait un formulaire avec `innerHTML`, mais dans ce test SonarQube n'a rien signalé et le pipeline est resté Passed.

J'ai donc fait un deuxième test en récupérant directement la valeur depuis l'URL :

```
const message = new URLSearchParams(window.location.search).get("message");
document.getElementById("message").innerHTML = message;
```

Cette fois, SonarQube a détecté une DOM XSS liée à une donnée utilisateur non nettoyée. L'issue était classée Blocker. Le Quality Gate est passé à Failed et le job d'analyse a échoué.

7. Résultat du pipeline

Le résultat visible dans GitLab correspond bien au fonctionnement attendu : certaines exécutions passent complètement, alors que la version contenant la faille XSS est bloquée.

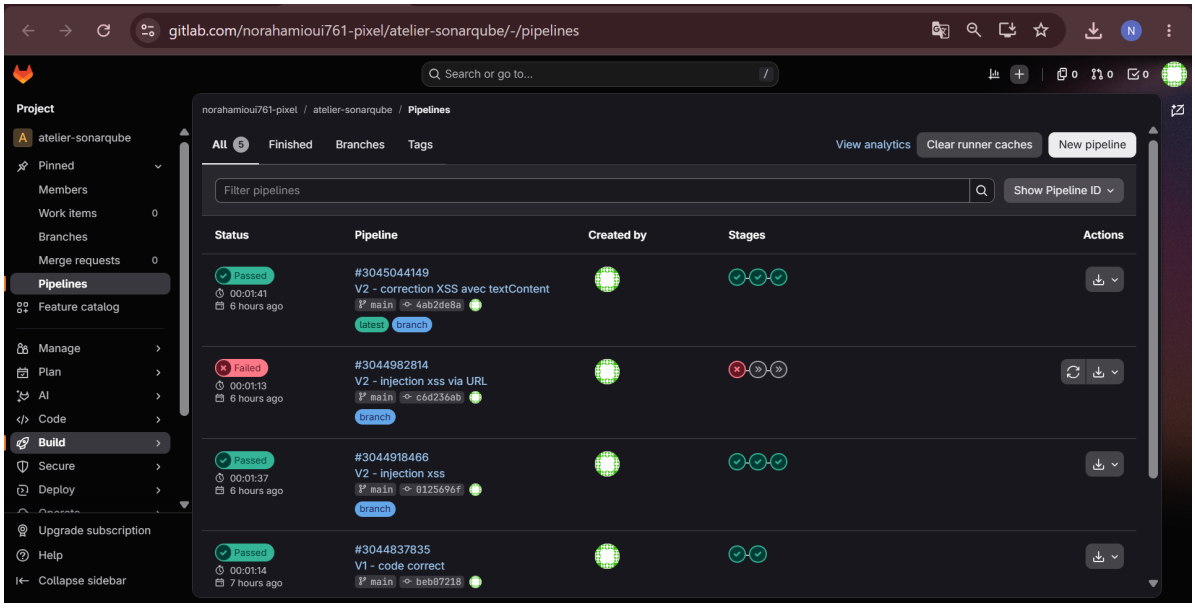


Figure 1 — Historique réel des pipelines GitLab : les exécutions Passed et Failed sont visibles.

Sur la capture, on voit notamment un pipeline Passed avec les trois étapes terminées correctement. On voit aussi un pipeline Failed correspondant au test XSS via l'URL : la première étape est en échec et les étapes suivantes ne sont pas exécutées.

8. Comment SonarQube a trouvé la faille

Le problème vient du chemin suivi par la donnée. La source est `window.location.search`, donc une donnée contrôlée par l'utilisateur. Cette donnée arrive ensuite dans `innerHTML`. Comme `innerHTML` interprète le contenu comme du HTML, un code JavaScript peut être exécuté dans le navigateur.

Dans ce cas, le pipeline joue donc le rôle de garde-fou : le code est analysé avant le déploiement et la publication est empêchée lorsque le Quality Gate est en échec.

9. Correction

Pour corriger la faille, j'ai gardé la lecture de l'URL mais remplacé `innerHTML` par `textContent` :

```
const message = new URLSearchParams(window.location.search).get("message");
document.getElementById("message").textContent = message;
```

Avec `textContent`, la valeur est affichée comme du texte et n'est pas interprétée comme du HTML. Après le push de cette correction, le pipeline repasse au vert, avec un Quality Gate Passed et 0 issue.

10. Historique des essais

Version	Résultat	Ce qui s'est passé
V1 - code correct	Passed	Analyse propre, Quality Gate validé et site publié.
V2 - injection XSS	Passed	Premier test avec formulaire + innerHTML : aucune issue détectée dans ce cas.
V2 - XSS via URL	Failed	DOM XSS détectée comme Blocker ; déploiement bloqué.
V2 - correction	Passed	Remplacement par textContent ; 0 issue et site de nouveau publié.

Capture du résultat Passed

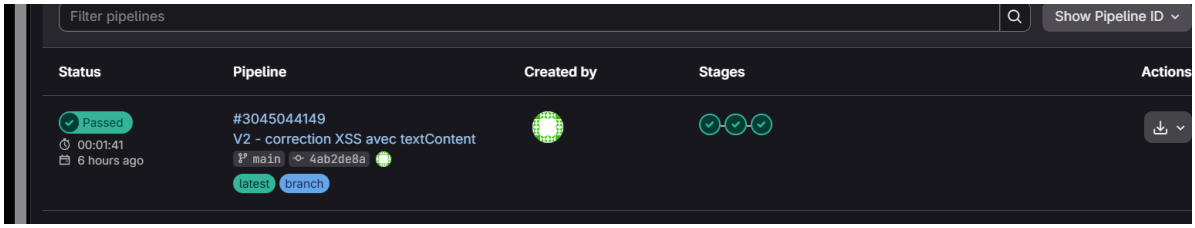


Figure 2 — Exemple d'un pipeline Passed visible dans GitLab.

Capture du résultat Failed

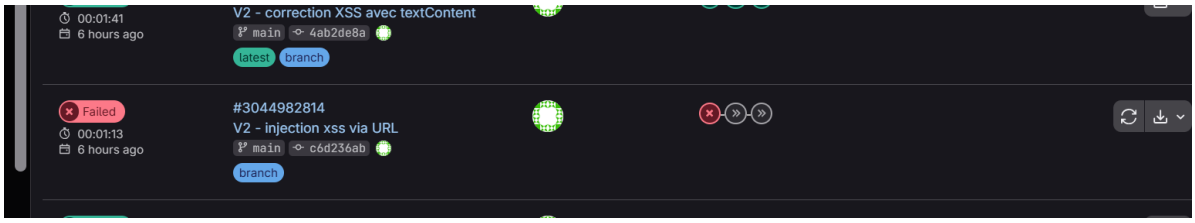


Figure 3 — Exemple du pipeline Failed lié au test XSS via URL.

11. Difficultés rencontrées

Pendant l'atelier, j'ai rencontré quelques problèmes qui m'ont obligé à refaire certaines étapes :

- Connexion SonarQube : le compte était d'abord relié à GitHub alors que le projet était sur GitLab. J'ai dû me déconnecter, me reconnecter avec GitLab et refaire l'import.
- Plan SonarQube : une période d'essai du plan Team était proposée. J'ai finalement utilisé le plan Free pour le projet public.
- Premier pipeline : GitLab demandait une vérification du compte avant d'autoriser l'exécution. Après la vérification, le pipeline a pu être relancé.
- Conflit d'analyse : j'ai désactivé l'Automatic Analysis de SonarQube pour laisser le pipeline CI gérer l'analyse.
- Première XSS non détectée : le test avec le formulaire n'a pas produit d'issue. En passant par une donnée venant directement de l'URL, SonarQube a détecté la DOM XSS.

12. Ce que j'ai retenu

- Un pipeline CI/CD peut empêcher automatiquement la publication d'un code qui ne respecte pas le Quality Gate.
- Les tokens et mots de passe doivent rester dans les variables CI/CD et ne pas être ajoutés au code.
- `innerHTML` avec des données non fiables peut créer une faille XSS ; `textContent` permet ici d'afficher la valeur sans l'interpréter.
- SonarQube est utile pour repérer des problèmes avant le déploiement, mais il ne remplace pas les tests ni la revue du code.
- La couverture de tests n'a pas été utilisée dans cet atelier, car le mini-projet ne contenait pas de tests automatisés.

13. Conclusion

Cet atelier m'a permis de voir concrètement le lien entre GitLab CI/CD et SonarQube. Le point le plus intéressant pour moi est le test V2 : une petite modification du JavaScript a suffi pour faire apparaître une faille DOM XSS et faire passer le pipeline de Passed à Failed. Après la correction avec textContent, le pipeline est redevenu vert. Cela montre bien l'intérêt d'ajouter un contrôle automatique avant le déploiement.

Annexe : les captures intégrées dans ce rapport proviennent de l'exécution réelle des pipelines GitLab du projet `atelier-sonarqube`.